

Annexe 3 : Conception d'un extracteur avec téléchargements HTTP simultanés

1. Contexte technique et prérequis de lecture	2
2. Architecture de la solution	2
A. Base existante, remise à niveau et restructuration	2
B. Le moteur hybride (Selenium vers Requests)	3
C. Multi-threading et nommage réseau	3
3. Preuves matérielles	4
4. Bilan	6

1. Contexte technique et prérequis de lecture

L'entreprise devait récupérer une nouvelle liste de documents sur un autre portail. Contrairement au premier projet qui utilisait **Documentum**, ce nouveau site propose de vrais liens de téléchargement direct. L'objectif était donc de créer un moteur beaucoup plus rapide pour traiter un grand volume de fichiers.

*(Note : Ce programme réutilise environ 80 % du code de mon premier outil. La lecture de l'**Annexe 2** est nécessaire pour comprendre la base du script, notamment la gestion du fichier de configuration, les sécurités de saisie utilisateur et les logs, qui sont identiques).*

2. Architecture de la solution

A. Base existante, remise à niveau et restructuration

Pour démarrer ce projet, j'ai repris la base de mon premier outil. Le problème, c'est que ce fichier unique approchait déjà des **1000 lignes**. Au lieu de rajouter du code par-dessus et de rendre les modifications compliquées pour la suite, j'ai décidé de prendre le temps de nettoyer l'existant.

Mon cadre de stage me permettait de privilégier la qualité technique au lieu de me dépêcher. J'ai donc utilisé ce temps pour me faire une vraie remise à niveau sur la **Programmation Orientée Objet (POO)**. Mon objectif était de créer une structure claire. Après cette formation, j'ai découpé mon programme : j'ai séparé la logique dans plusieurs fichiers **Python** et j'ai créé des classes comme "**ConfigManager**" et "**ConfigQuestions**". J'ai utilisé des méthodes statiques (**@staticmethod**) pour gérer mes vérifications. Comme ça, le code principal est devenu très léger, et je peux réutiliser chaque fichier pour d'autres projets.

B. Le moteur hybride (Selenium vers Requests)

Sur ce portail, la page de connexion est identique à 99 % à celle du premier projet. J'ai donc fait le choix le plus pratique : j'ai réutilisé mon code **Selenium** existant pour cette étape.

Le programme ouvre le navigateur en mode invisible (*headless*) uniquement pour passer l'authentification. Il extrait ensuite les **cookies** de la session (`driver.get_cookies()`). Il ferme complètement **Selenium** et donne ces **cookies** à la librairie "**requests**". J'ai mis en place cette transition pour des raisons de performances. Le script gère les téléchargements en **HTTP** classique car cela consomme beaucoup moins de **RAM** et est aussi beaucoup plus rapide (pas de chargement de page). Il injecte aussi le **User-Agent** d'un vrai navigateur dans l'en-tête pour éviter d'être bloqué par le serveur.

C. Multi-threading et nommage réseau

Le fichier **Excel** de départ ne donne pas les **URL** complètes, mais seulement des numéros de documents. Le script reconstruit les liens de téléchargement dynamiquement.

Les requêtes simultanées : Comme ce sont des téléchargements directs, j'ai utilisé la librairie "**concurrent.futures**". Le programme lance un **ThreadPoolExecutor** avec **10 travailleurs en parallèle**. Pour supporter cette charge réseau sans planter, j'ai configuré un adaptateur **HTTPAdapter** pour gérer proprement le **pool de connexions**.

Extraction des noms : Comme l'URL ne contient pas le nom du fichier, le programme lit l'en-tête réseau "**Content-Disposition**" dans la réponse du serveur. Il récupère le vrai nom du document (**PDF** ou **ZIP**), il supprime les caractères interdits par **Windows**, et sauvegarde le fichier sur le disque.

3. Preuves matérielles

Restructuration du code en modules indépendants et utilisation de classes :

```
# %% Message si l'utilisateur répond "Oui" et "Non" dans le même message
@staticmethod
def ambiguReponse():
    print("Réponse ambiguë, veuillez être plus précis.")

# %% Demande si l'utilisateur souhaite reconfigurer la config actuelle (si il y en a déjà une)
@staticmethod
def demander_reconfiguration(askReconfig: bool) -> bool:
    wantConfig = False
    while askReconfig:
        confirmDelay = 10
        phrase = f'Souhaitez-vous reconfigurer le dossier de sortie et la liste des fichiers Excel ? (Oui / Non) ("Non" sera au
        reconfigExcelsQuestion = ConfigQuestions.input_with_timeout(phrase, confirmDelay)
        reconfigExcelsReponse = ConfigQuestions.choixOuiNon(reconfigExcelsQuestion)
        if reconfigExcelsReponse == "OUI":
            wantConfig = True
            break
        elif reconfigExcelsReponse == "NON":
            wantConfig = False
            break
        elif reconfigExcelsReponse == "CONFLIT":
            ConfigManager.ambiguReponse()
        else:
            print("Veuillez entrer une réponse valide.")
    return wantConfig
```

Figure 1 : Nouvelle structure modulaire du code (POO).

Méthode de récupération et de transfert des données de session :

```
# %% Récupération des cookies de Selenium et on les applique à Request, on ferme la fenêtre juste après
print("Préparation de la mise en route des téléchargements...")
selenium_cookies = driver.get_cookies()
cookies_dict = {c['name']: c['value'] for c in selenium_cookies}
vrai_user_agent = driver.execute_script("return navigator.userAgent;")

session = requests.Session()
taille_pool = 20
adapter = HTTPAdapter(pool_connections=taille_pool, pool_maxsize=taille_pool)
session.mount('http://', adapter)
session.mount('https://', adapter)

session.cookies.update(cookies_dict)
session.headers.update({"User-Agent": vrai_user_agent})
# On a plus besoin du navigateur, on le ferme
driver.quit()

# Il va afficher le résumé de l'opération si le programme se termine
stopDuringConfig = False
```

Figure 2 : Moteur hybride et injection des cookies dans la session HTTP.

Lancement des travailleurs en parallèle pour accélérer le traitement :

```
# %% Téléchargement multi-tâche
print("Démarrage des téléchargements...")

tasks = liens_uniques
temps_debut = time.time()
with ThreadPoolExecutor(max_workers=10) as executor:
    results = list(executor.map(download_file, tasks))
```

Figure 3 : Implémentation du multi-threading et gestion du pool de connexions réseau.

Extraction dynamique du nom du fichier envoyé par le serveur :

```
def download_file(args):
    link, numBrut = args
    returnText = None
    statut_final = "ECHEC"

    try:
        response = session.get(link, stream=True, timeout=120, verify=False)

        if response.status_code == 200:
            cd_header = response.headers.get('Content-Disposition')
            filename = re.findall('filename="?([^"]+)"?', cd_header)[0] if cd_header and 'Filename=' in cd_header else f"document_{numBrut}.pdf"
            clean_filename = re.sub(r'[\\/*?:"<>|]', "_", filename)
            chemin_complet = os.path.join(target_dir, clean_filename)

            with open(chemin_complet, 'wb') as f:
                for chunk in response.iter_content(chunk_size=8192):
                    f.write(chunk)

            returnText = f"[{numBrut}] OK : {clean_filename}"

            nomLowercase = clean_filename.lower()
            if nomLowercase.endswith('.zip'):
                statut_final = "ZIP"
            elif nomLowercase.endswith('.pdf'):
                statut_final = "PDF"
            else:
                statut_final = "AUTRE_FORMAT"
        else:
            returnText = f"[{numBrut}] Erreur : {response.status_code} : {link}"
            statut_final = "ERREUR"

    except Exception as e:
        returnText = f"[{numBrut}] Échec : {e}"
        statut_final = "ECHEC"

    print(returnText, flush=True)

    enregistrar_statut(link, statut_final, numBrut, response.status_code)

    return statut_final
```

Figure 4 : Lecture des en-têtes HTTP pour le nommage des fichiers.

4. Bilan

À la fin du traitement, le programme vérifie automatiquement son travail. Il compare mathématiquement le nombre de liens scannés dans le fichier **Excel** avec le nombre de fichiers **PDF** physiquement sauvegardés sur le disque. S'il y a la moindre différence, il affiche une alerte dans la console et trace l'erreur dans le fichier de **log**. C'est un outil autonome et fiable qui récupère des centaines de documents en quelques minutes.